

Assessment Document

Multivariate Time-Series Prediction Using Deep Learning

Candidate Name: D.R.R.C.Dasanayake

Date: 2026/06/07

Table of Contents

1. Introduction.....	4
1.1. Brief overview of the problem.....	4
1.2 Core Objective	4
1.3 Assessment Scope	4
2. Data Insights	5
2.1 Key visualizations	6
2.2 Key findings.....	8
3. Data preprocessing.....	8
3.1 Missing Values	8
3.2 Outlier Handling	8
3.3 Feature Scaling.....	9
3.4 Train-Test Split.....	9
4. Feature engineering.....	9
4.1 Time-Based Features.....	9
4.2 Lag Features.....	10
4.3 Rolling Statistics	10
4.4 Interaction Features.....	10
4.5 Feature Selection.....	10
Selected Features	10
5. Model Design.....	11
5.1 Deep Learning Architectures	11
5.2 Architectural Design Choices	12
6. Results.....	12
6.1 Evaluation Metrics	12
6.2 Model Performance Comparison	13
6.3 Key Visualizations	14
6.4 Model Comparison and Analysis	17
6.5 Residual Analysis and Predicted vs Actual Behavior	17

7. Model Optimization	18
7.1 Optimization Techniques Applied.....	18
7.2 Ablation Study: Optimizers and Activation Functions	20
8. Challenges and Solutions	21
9. Conclusion and Future Work	23
9.1 Key Findings	23
9.2 Future Work	23
10. References	24
11. Declaration of AI Tools	24

1. Introduction

1.1. Brief overview of the problem

Energy consumption prediction is a critical challenge in modern smart building management, enabling intelligent automation, cost reduction, and environmental sustainability. This report presents a comprehensive study on predicting appliance energy consumption using the Appliance Energy Prediction Dataset - a multivariate time-series dataset collected at 10-minute intervals across a residential building over a five-month period (January to May 2016).

The project addresses the full machine learning pipeline:

Exploratory data analysis (EDA), data preprocessing, feature engineering, deep learning model design and training, hyperparameter optimisation, and comprehensive evaluation. Three deep learning architectures are explored: **Long Short-Term Memory (LSTM)**, **Gated Recurrent Unit (GRU)** and a **CNN-LSTM hybrid** and benchmarked against classical baseline models (**Linear Regression** and **Random Forest**).

1.2 Core Objective

Predict the energy consumption (Wh) of household appliances at 10-minute resolution using environmental, temporal, and lagged features. Minimise MAE and RMSE on an unseen held-out test set whilst avoiding overfitting through systematic regularisation and early stopping.

1.3 Assessment Scope

Data Understanding - Conduct thorough EDA to uncover temporal patterns, correlations, and anomalies.

Preprocessing - Handle missing values, outliers, scaling, and temporal splitting.

Feature Engineering - Create time-based, rolling, lagged, and interaction features; perform feature selection.

Model Development - Design and compare LSTM, GRU, and CNN-LSTM architectures against baselines.

Optimisation - Apply hyperparameter tuning, dropout, and early stopping to maximise accuracy.

Documentation - Produce a professional, reproducible report with justified design decisions

2. Data Insights

The Appliance Energy Prediction dataset contains 19,735 observations collected at 10-minute intervals over approximately 4.5 months. The target variable, **Appliances** energy consumption, exhibits a right-skewed distribution, where most observations correspond to low-to-moderate energy usage, while occasional high-consumption events reach up to 1,080 Wh. These extreme values represent genuine appliance usage behavior and were therefore retained for model training and evaluation (**Target Distribution Histogram**).

Exploratory Data Analysis revealed clear temporal and seasonal patterns in household energy consumption. Daily seasonality was evident, with energy usage increasing during morning and evening hours and decreasing overnight (**Energy Consumption Trend Line Chart and Hourly Energy Consumption Profile**). Weekly patterns showed slightly higher average consumption during weekends compared to weekdays, indicating the influence of occupancy and household activities on energy demand.

Hourly profile and boxplot analyses further highlighted a bimodal consumption pattern, with distinct peaks occurring during morning and evening periods (**refer to Box Plot by Hour**). The variability of energy usage was also observed to increase during peak activity periods, suggesting greater uncertainty in appliance operation during these times.

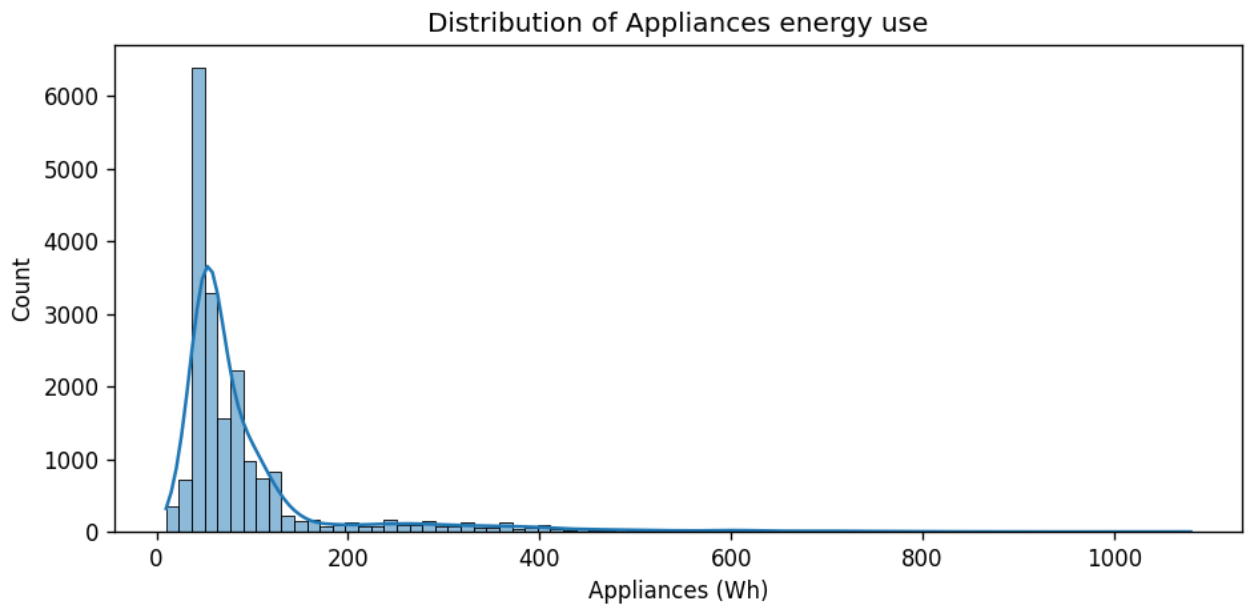
The correlation heatmap was used to examine relationships among environmental variables and appliance energy consumption (**illustrated in the Correlation Heatmap**). While no single sensor feature exhibited a strong linear relationship with the target variable, indoor temperature, humidity, weather conditions, and time-related variables collectively contributed valuable predictive information. Strong correlations among several temperature and humidity sensors indicated the presence of feature redundancy and multicollinearity.

Feature importance analysis showed that lagged energy consumption variables and rolling statistical features were the most influential predictors, confirming the strong temporal dependency of appliance energy usage. These findings justified the use of time-series feature engineering techniques and sequence-based deep learning models, including LSTM, GRU, and CNN-LSTM architectures.

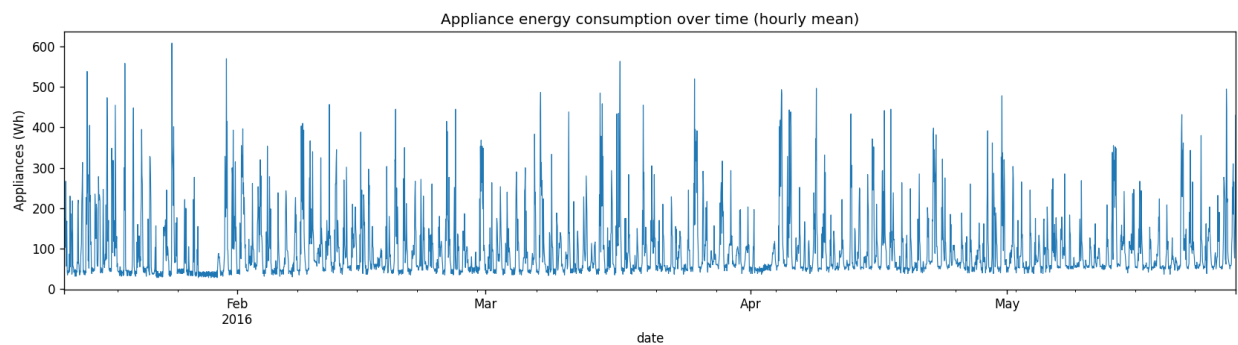
Overall, the EDA identified significant temporal dependencies, seasonal trends, and non-linear relationships within the dataset, providing critical insights that guided the preprocessing, feature engineering, and model development stages of the project.

2.1 Key visualizations

Distribution of Appliances Energy Consumption

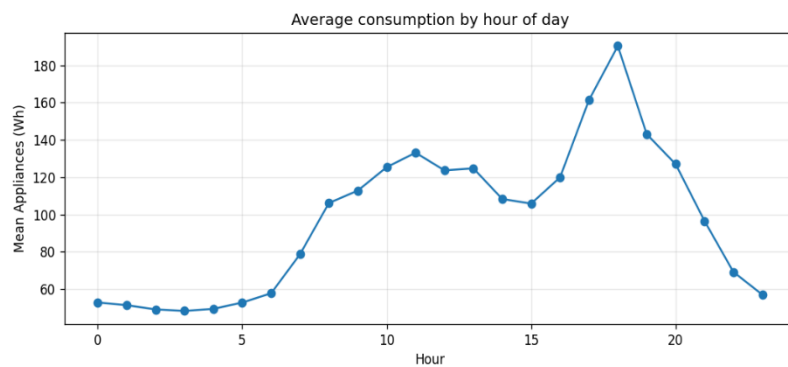


Energy Consumption Over Time

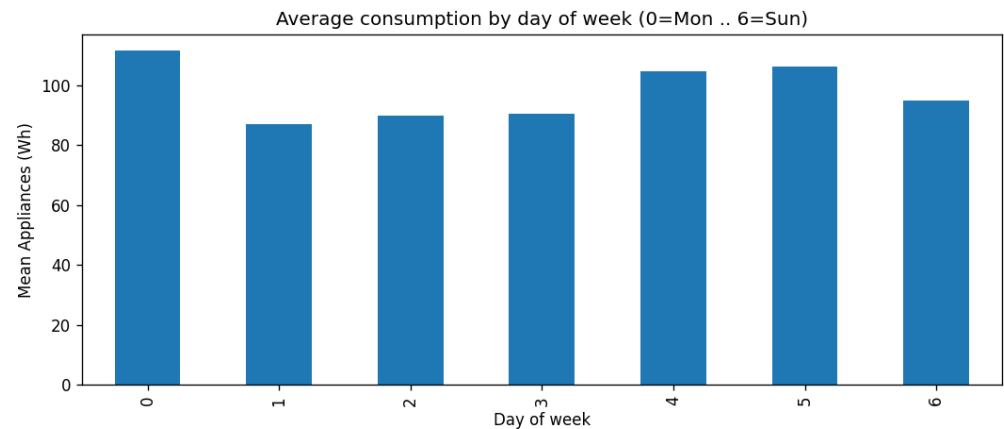


Temporal and Seasonal Patterns

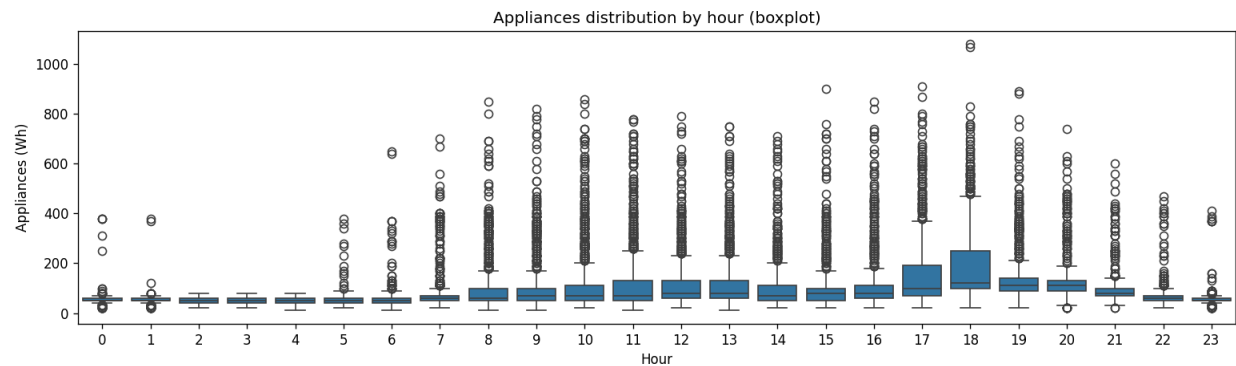
Hourly Profile (Daily Seasonality)



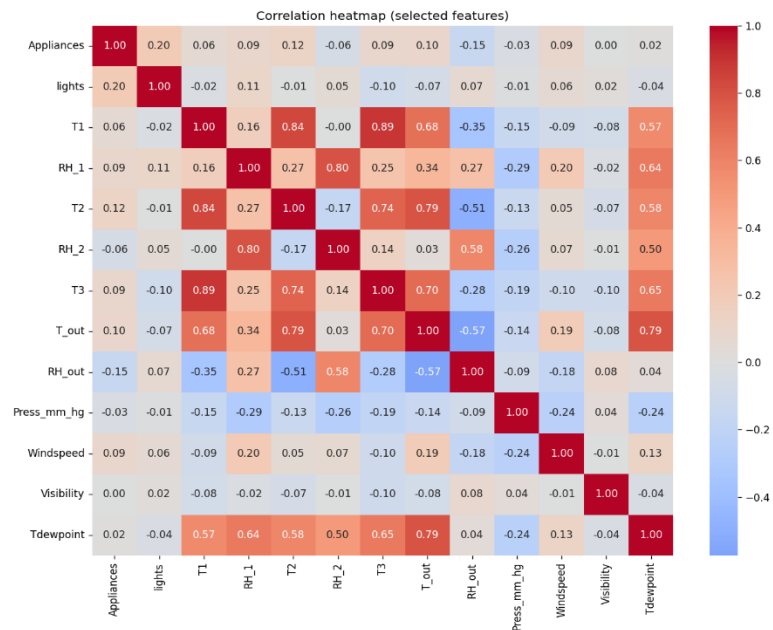
Day-of-Week Profile (Weekly Seasonality)



Hourly Variability (Boxplot Analysis)



Correlation Heatmap



2.2 Key findings

- **Skewed energy usage:** Appliance energy consumption is right-skewed, with most values low and occasional high spikes up to 1,080 Wh (real usage events).
- **Daily and weekly patterns:** Clear daily seasonality exists with peaks in morning and evening, while weekends show slightly higher consumption than weekdays.
- **Bimodal behavior:** Energy usage shows a bimodal pattern with distinct morning and evening peaks, and higher variability during peak hours.
- **Weak direct correlations:** No single sensor strongly predicts energy use, but combined environmental factors (temperature, humidity, weather, time) are important.
- **Multicollinearity present:** Strong correlations among sensor features indicate redundancy in temperature and humidity measurements.
- **Lag features are most important:** Past energy usage (lagged values) and rolling statistics are the strongest predictors of future consumption.
- **Strong temporal dependency:** Energy usage is highly time-dependent, justifying time-series feature engineering.

3. Data preprocessing

All preprocessing steps were implemented in `src/data_preprocessing.py` to prepare the dataset for machine learning and deep learning models while ensuring no data leakage.

3.1 Missing Values

Missing values were handled using time-series aware methods:

- **Time-based interpolation:** `interpolate(method="time")` was used to preserve temporal trends.
- **Edge filling:** Remaining missing values were filled using forward and backward fill.

This maintains continuity without distorting time patterns.

3.2 Outlier Handling

Outliers were treated using the IQR method:

- **Environmental features:** Outliers were winsorized using IQR boundaries to remove sensor noise.
- **Target variable:** No outlier removal was applied, as energy spikes represent real consumption events.

- **Data integrity:** No rows were removed, preserving continuous time steps for sequence modeling.

3.3 Feature Scaling

Feature scaling was applied to stabilize model training:

- **Standardization (Z-score):**

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

- Applied to all numeric features for zero mean and unit variance.
- **Scaler fitted only on training data** to prevent data leakage.
- A separate scaler was used for the target variable, allowing inverse transformation for evaluation in Wh.

3.4 Train-Test Split

A chronological split was used:

- **80% training / 20% testing** based on time order
- Avoids data leakage caused by random splitting
- Simulates real-world forecasting where future data is unknown

4. Feature engineering

Feature engineering was performed to capture temporal patterns, historical energy consumption behavior, and environmental influences that affect appliance energy usage.

4.1 Time-Based Features

The timestamp was transformed into several temporal attributes, including hour of day, day of week, month, and weekend indicators. To preserve the cyclical nature of time, sine and cosine transformations were applied to hourly and weekly features.

Justification: Energy consumption follows strong daily and weekly patterns. Cyclical encoding enables the model to correctly interpret temporal relationships such as the similarity between 23:00 and 00:00.

4.2 Lag Features

Lagged versions of the target variable (e.g., 10, 20, 30, and 60 minutes prior) were created using autocorrelation analysis.

Justification: Previous energy consumption values are strong predictors of future consumption due to the autoregressive nature of time-series data.

4.3 Rolling Statistics

Rolling mean and rolling standard deviation features were computed over 1-hour and 3-hour windows.

Justification: These features capture short-term consumption trends and variability, allowing the model to understand recent energy usage patterns.

4.4 Interaction Features

Interaction terms were generated between selected temperature and humidity variables.

Justification: Appliance usage is often influenced by combined environmental conditions rather than individual variables alone. Interaction features help capture these non-linear relationships.

4.5 Feature Selection

A Random Forest model was used to rank feature importance, and the top 25 most informative features were selected for model training.

Justification: Feature selection reduces dimensionality, removes redundant information, improves computational efficiency, and helps prevent overfitting.

Selected Features

The most important features included lagged energy consumption variables, rolling statistics, time-based features, atmospheric pressure, indoor temperature sensors, and temperature-humidity interaction terms. Among these, the lag-1 feature showed the highest predictive importance, confirming the strong temporal dependency of appliance energy consumption.

5. Model Design

To capture temporal dependencies and complex patterns in household energy consumption, three deep learning architectures were developed and evaluated using **PyTorch**: Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU) and CNN-LSTM.

5.1 Deep Learning Architectures

- **LSTM Regressor**

The LSTM model consists of two stacked LSTM layers with 64 hidden units, followed by a fully connected prediction head. The final hidden state is passed through a dense layer ($64 \rightarrow 32$), a **ReLU** activation function, a dropout layer (0.3), and an output layer ($32 \rightarrow 1$) to predict appliance energy consumption.

The LSTM architecture was selected because it can effectively learn long-term temporal dependencies while mitigating the vanishing gradient problem commonly encountered in traditional recurrent neural networks.

- **GRU Regressor**

The GRU model follows a similar architecture with two stacked GRU layers and 64 hidden units. The output is processed through a dense layer, **ReLU** activation, dropout regularization, and a final regression layer.

GRU was chosen because it requires fewer parameters than LSTM while maintaining strong sequence-learning capabilities, resulting in faster training and improved generalization on medium-sized datasets.

- **CNN-LSTM Regressor**

The CNN-LSTM architecture combines two one-dimensional convolutional layers with an LSTM layer. The convolutional layers extract local temporal patterns and sensor interactions, while the LSTM captures long-term sequential dependencies. The extracted features are then passed through fully connected layers to generate the final prediction.

This hybrid approach was designed to capture both short-term fluctuations and long-term temporal trends present in appliance energy consumption data.

5.2 Architectural Design Choices

Sequence Length (12 Timesteps):

A sequence length of 12 timesteps (2 hours of historical data) was selected to provide sufficient temporal context while maintaining computational efficiency.

Recurrent Layers (2 Layers):

Two recurrent layers were used to enable the models to learn both low-level temporal patterns and higher-level seasonal trends.

Hidden Units (64):

A hidden size of 64 provided a balance between model complexity and generalization performance, avoiding both underfitting and excessive overfitting.

Activation Function (ReLU):

ReLU was used in the dense layers because it is computationally efficient and helps alleviate vanishing gradient issues. Alternative activation functions such as Tanh and GELU were also evaluated during experimentation.

Optimizers (Adam and RMSprop):

Adaptive optimizers were employed because they automatically adjust learning rates during training, leading to faster convergence and more stable optimization of recurrent neural networks.

Loss Function (Mean Squared Error):

Mean Squared Error (MSE) was selected as the loss function because it strongly penalizes large prediction errors. Since appliance energy consumption contains occasional high-energy spikes, MSE encourages the model to learn these important peak-demand events more effectively.

6. Results

This section presents the evaluation of all implemented models using a chronologically split dataset (80% training and 20% testing). The split ensures temporal integrity by using the first ~3.6 months for training and the final ~0.9 months for testing, thereby preventing temporal data leakage.

6.1 Evaluation Metrics

All model predictions were inverse-transformed back to the original energy consumption scale (Wh) before evaluation. The following standard regression metrics were used:

- **Mean Absolute Error (MAE):** Measures the average magnitude of prediction errors, regardless of direction.

- **Root Mean Squared Error (RMSE):** Penalizes larger deviations more heavily and reflects model sensitivity to peak load variations.
- **Mean Absolute Percentage Error (MAPE):** Expresses prediction error as a percentage of actual values, enabling relative performance comparison.
- **Coefficient of Determination (R^2):** Indicates how well the model explains variance in the target variable.

6.2 Model Performance Comparison

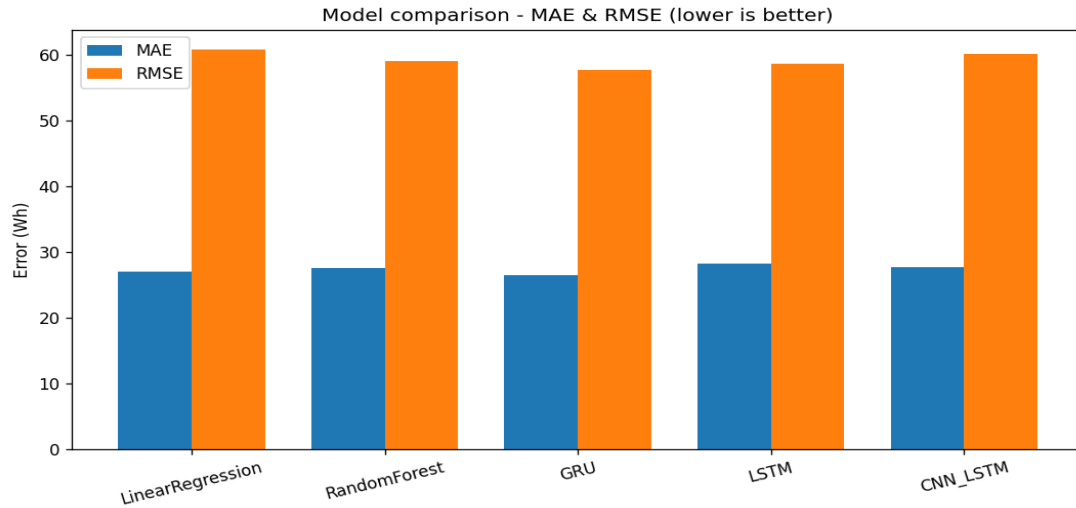
The table below summarizes the performance of all evaluated models on the test dataset.

Model Class	Model	MAE (Wh)	RMSE (Wh)	MAPE (%)	R^2
Baseline	LinearRegression	26.9831	60.7103	24.3204	0.5544
Baseline	RandomForest	27.5267	59.0120	25.2235	0.5790
Deep Sequence	CNN-LSTM	27.6646	60.0125	25.7037	0.5450
Deep Sequence	LSTM	28.2336	58.5237	26.2229	0.5816
Deep Sequence	GRU (Best)	26.4739	57.6061	24.2794	0.5807

6.3 Key Visualizations

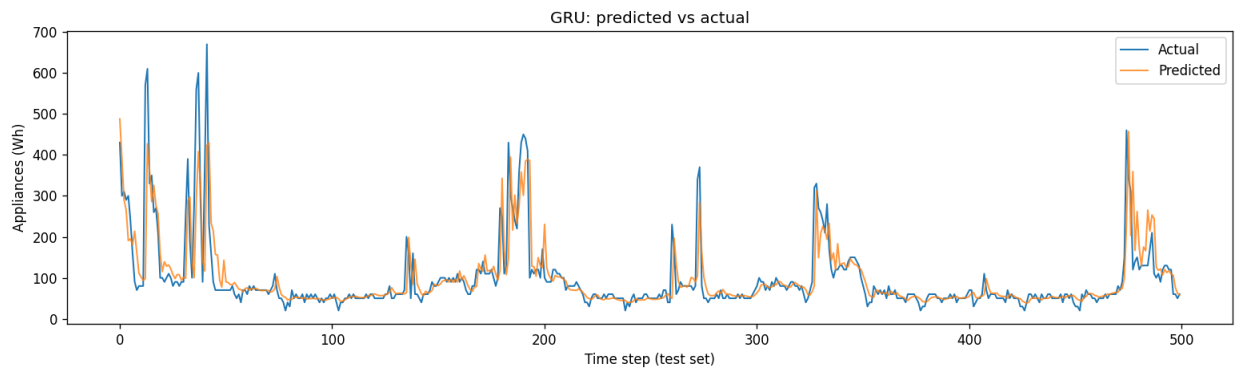
The following figures were generated to support model evaluation and interpretability

Bar chart comparing MAE, RMSE, and R^2 across all models.

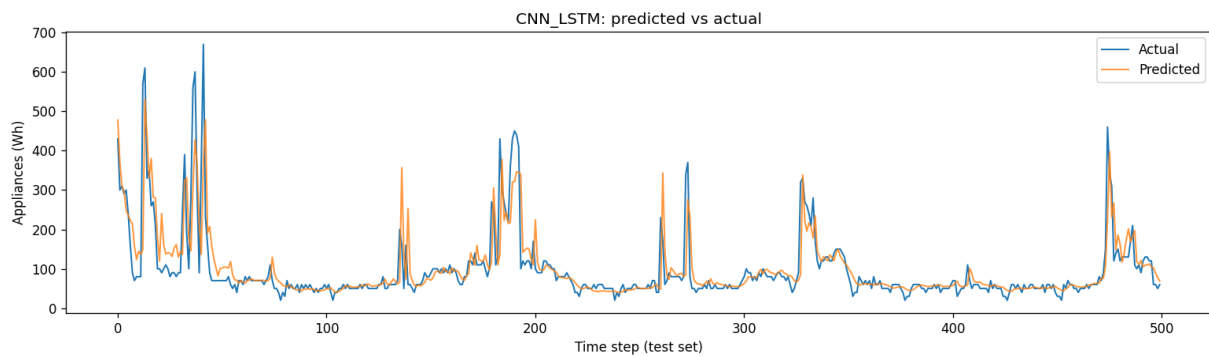


Predicted vs. Actual Scatter Plots

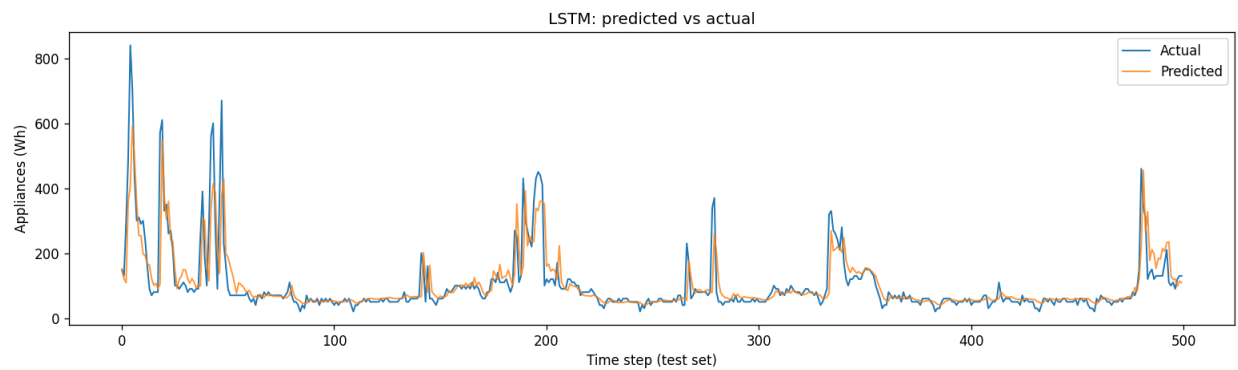
Scatter plot of predicted vs. actual values for the best GRU model.



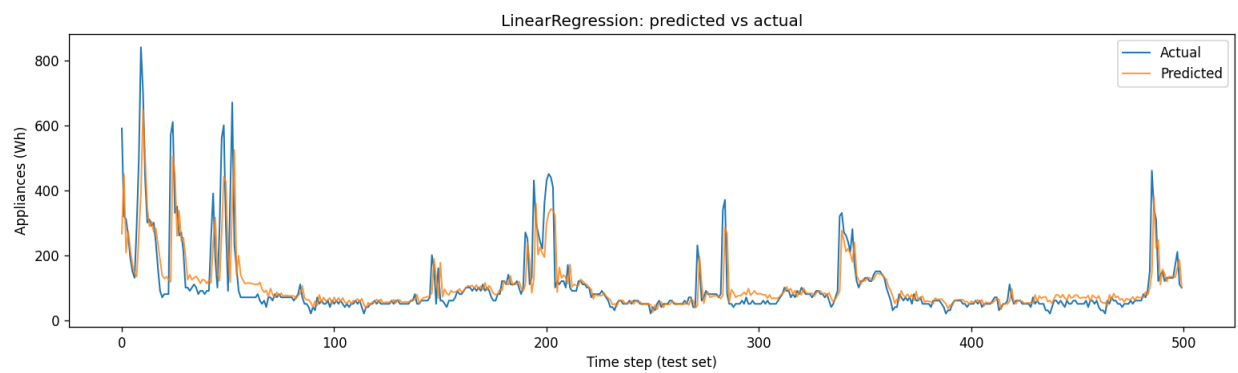
Predicted vs Actual Energy Consumption (CNN-LSTM Model)



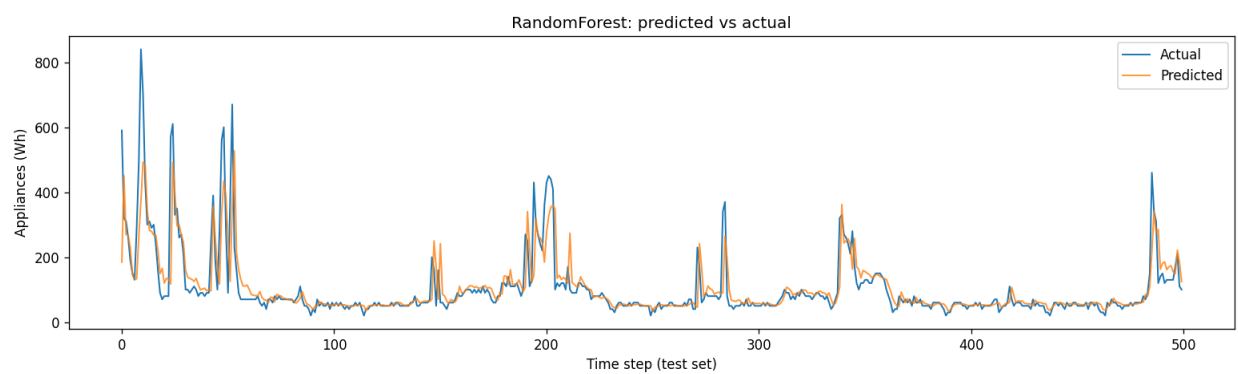
Predicted vs Actual Energy Consumption (LSTM Model)



Predicted vs Actual Energy Consumption (Linear Regression Model)

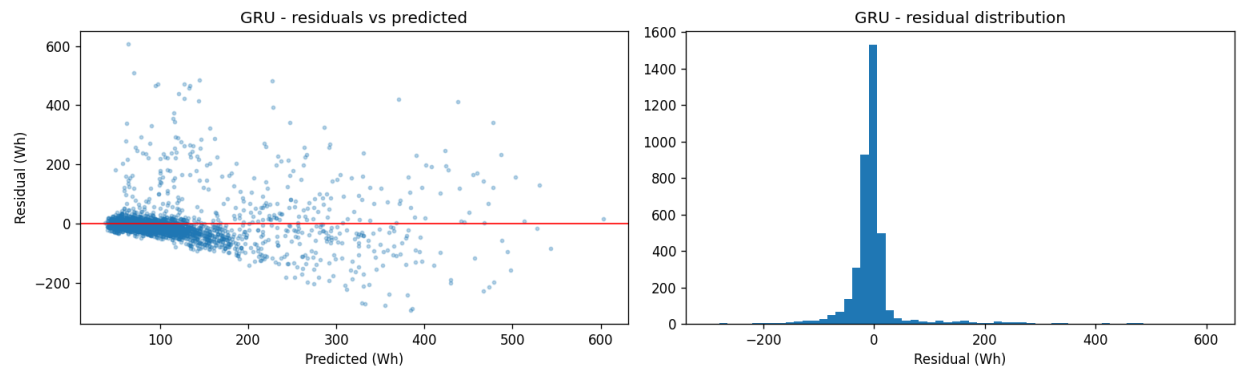


Predicted vs Actual Energy Consumption (Random Forest Model)

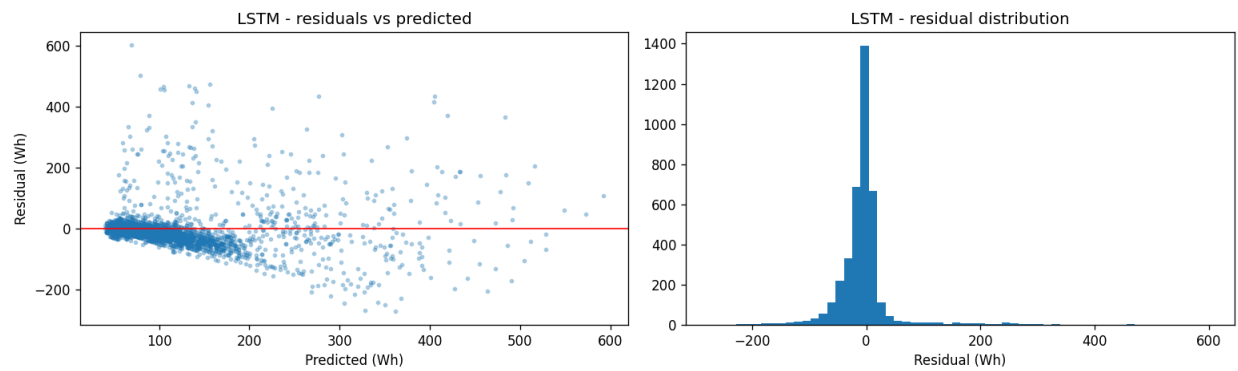


Residual Analysis Plots:

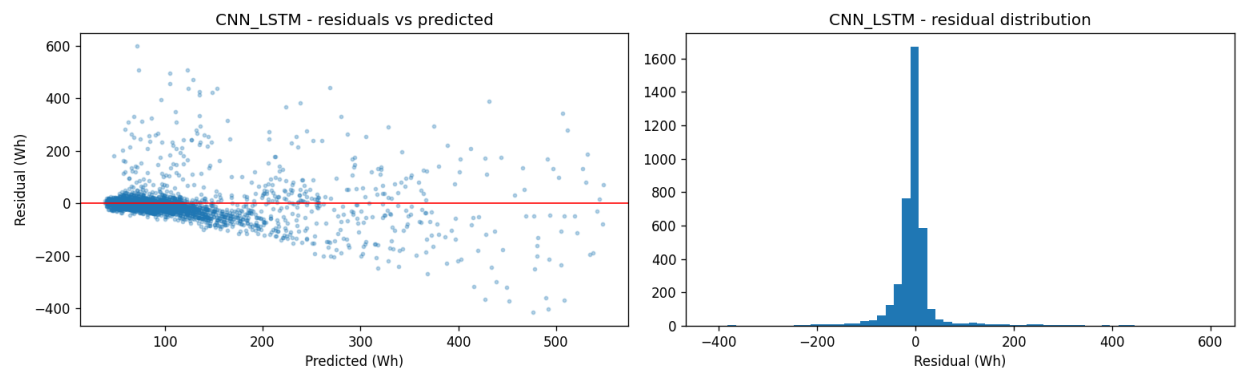
Residual analysis plot for the best GRU model.



Residual Analysis of LSTM Model Predictions



Residual Analysis of CNN-LSTM Model Predictions



6.4 Model Comparison and Analysis

1. **GRU Superiority:** The GRU model achieves the lowest MAE (**26.47 Wh**) and RMSE (**57.61 Wh**), combined with a strong R^2 score (**0.5807**). Its reduced parameter count and efficient gating structure allow it to capture the autoregressive energy dynamics without overfitting.
2. **LSTM Performance:** The LSTM achieves the highest overall variance explanation (R^2 of **0.5816**) and a competitive RMSE of **58.52 Wh**. However, it suffers from a slightly higher MAE (**28.23 Wh**) and MAPE (**26.22%**), indicating a tendency to make larger average errors on baseline loads compared to the GRU.
3. **Weakness of CNN-LSTM Hybrid:** The hybrid CNN-LSTM model underperforms, with an R^2 of 0.5450 and RMSE of 60.01 Wh (ranking below the Linear Regression baseline on some metrics). Because the dataset's target is heavily dominated by the immediate lag-1 autoregressive signal (`Appliances_lag_1` has a Random Forest feature importance of 0.6059), the 1D CNN's temporal convolutions smooth out the sequence. This smoothing dilutes the sharp, immediate predictive power of the final timestep's lag, degrading next-step accuracy.
4. **Random Forest vs. Deep Learning:** The constrained Random Forest baseline performs remarkably well ($R^2 = 0.5790$, RMSE = 59.01 Wh), closely matching the deep learning models. This is due to the tabular nature of the engineered lag features which tree ensembles exploit effectively. However, the GRU outperforms it on all metrics, showing the benefit of learning continuous recurrent states.

6.5 Residual Analysis and Predicted vs Actual Behavior

- **High Density at Low Consumption:** In the predicted vs. actual plots (e.g., `pred_vs_actual_gru.png`), there is a tight clustering of points along the diagonal for values below 100 Wh, demonstrating that all models accurately capture the baseline household energy usage.
- **Spike Underestimation:** For high-energy consumption spikes (above 400 Wh), all models exhibit a tendency to underpredict. This is a characteristic of models optimized with MSE loss: since energy spikes are rapid, stochastic events (e.g., a washing machine running for 30 minutes and then stopping), a model attempting to minimize squared error will predict the conditional mean (a smoothed curve) rather than committing to a high-magnitude peak that might shift in time.
- **Residual Heteroscedasticity:** Residual plots (e.g., `residuals_gru.png`) show that errors are centered around zero but expand in variance as the actual value increases. The right-skewed, heavy-tailed nature of the target variable drives this heteroscedasticity.

7. Model Optimization

To enhance the predictive performance of deep sequence learning models, a structured optimization pipeline was implemented. This includes hyperparameter tuning, training stabilization techniques, and ablation studies to analyze the impact of key design choices.

7.1 Optimization Techniques Applied

Hyperparameter Optimization (Random and Grid Search)

A systematic tuning framework (`tune.py`) was developed to explore an extensive hyperparameter space using **Random Search**, followed by a refined **Grid Search** over the most promising regions.

The best-performing configuration was identified for the GRU model with the following parameters:

- Sequence length: **12** (approximately 2-hour input window)
- Hidden size: **64**
- Number of layers: **2**
- Dropout rate: **0.3**
- Learning rate: **0.001**
- Batch size: **128**
- Optimizer: **RMSprop**
- Activation function: **ReLU (MLP head)**

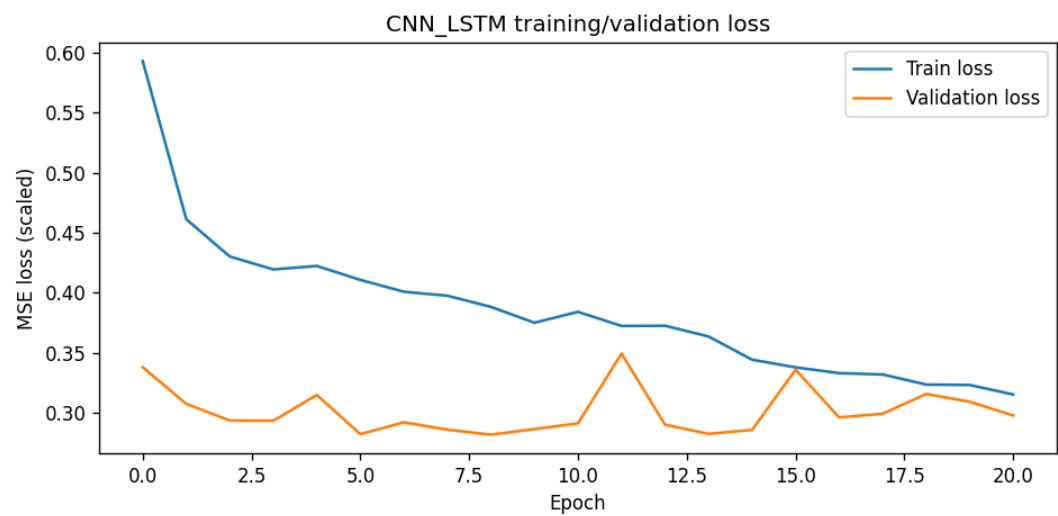
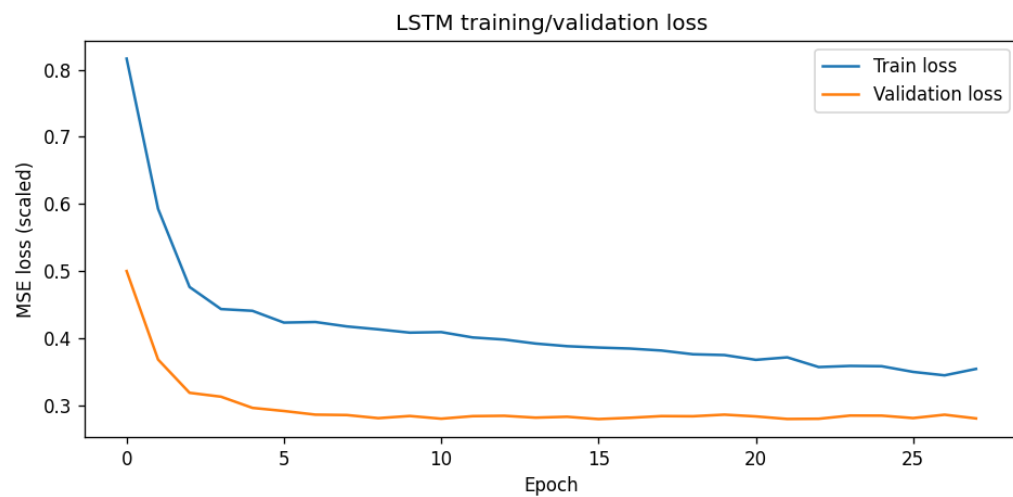
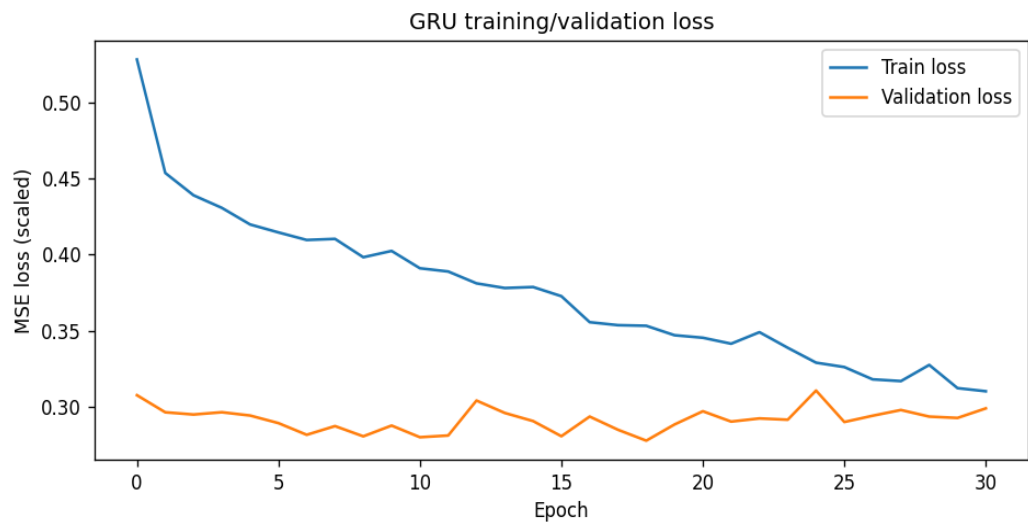
Validation Performance:

- MAE: **27.5895 Wh**
- RMSE: **59.4303 Wh**
- R^2 : **0.6216**

Early Stopping

To prevent overfitting, early stopping was applied using a hold-out validation split (last 15% of the training data). Training was terminated if validation loss did not improve for **12 consecutive epochs (patience = 12)**. The best-performing model weights were restored automatically.

Training and Validation Loss Curves



Learning Rate Scheduling

A ReduceLROnPlateau scheduler was used to improve convergence stability. If validation loss plateaued for 4 epochs, the learning rate was reduced by a factor of 0.5, enabling fine-grained optimization during later training stages.

Gradient Clipping

To mitigate gradient explosion issues commonly observed in recurrent networks, gradient norms were clipped to a maximum value of 1.0 using `torch.nn.utils.clip_grad_norm_`. This ensured stable backpropagation through time for both GRU and LSTM architectures.

7.2 Ablation Study: Optimizers and Activation Functions

An ablation study was conducted using the optimal GRU configuration to evaluate the effect of different optimizers and activation functions on validation performance.

(1) Optimizer Ablation (Activation = ReLU)

- **Adam:**
MAE = 27.2201 Wh, RMSE = 60.9851 Wh, $R^2 = \mathbf{0.6016}$ (23 epochs)
- **RMSprop:**
MAE = 27.6162 Wh, RMSE = 60.8878 Wh, $R^2 = \mathbf{0.6028}$ (25 epochs)
- **SGD (Momentum):**
MAE = 29.6355 Wh, RMSE = 68.4569 Wh, $R^2 = \mathbf{0.4979}$ (25 epochs)

Insight:

Adaptive optimizers such as **Adam** and **RMSprop** significantly outperform SGD. SGD shows degraded performance due to its inability to adapt learning rates dynamically across parameters, leading to poor convergence on complex recurrent loss landscapes.

(2) Activation Function Ablation (Optimizer = Adam)

- **ReLU:**
MAE = 27.2201 Wh, RMSE = 60.9851 Wh, $R^2 = \mathbf{0.6016}$
- **GELU:**
MAE = 27.0173 Wh, RMSE = 61.6519 Wh, $R^2 = \mathbf{0.5928}$
- **Tanh:**
MAE = 28.1107 Wh, RMSE = 61.7278 Wh, $R^2 = \mathbf{0.5918}$

Insight:

ReLU and GELU outperform Tanh in terms of R^2 and stability. Their non-saturating gradient behavior allows more efficient learning in the dense prediction head, whereas Tanh suffers from saturation effects that reduce gradient flow during training.

8. Challenges and Solutions

Overfitting in Baseline Random Forest

Challenge:

The initial unconstrained Random Forest model exhibited severe overfitting. Although the training error was low, the test-set performance collapsed to an R^2 of approximately 0.05. This occurred because the highly informative autoregressive feature (`Appliances_lag_1`) enabled the model to effectively memorize the target behavior rather than generalize from underlying patterns.

Solution:

Regularization strategies were introduced to improve generalization:

- Increased `min_samples_leaf` = 20
- Limited feature sampling with `max_features` = 0.5

These constraints forced the model to consider broader feature interactions (e.g, environmental and temporal variables), resulting in improved generalization and recovery of test performance to $R^2 = 0.5790$.

Temporal Data Leakage in Preprocessing

Challenge:

Improper preprocessing (e.g, global scaling or random splitting) introduced temporal leakage, where future information inadvertently influenced training.

Solution:

A strict chronological split (80% training, 20% testing) was enforced. Additionally:

- Scalers were fit **only on training data**
- Transformations were applied separately to training and test sets
- Feature selection was restricted to the training partition only

This ensured a realistic time-series evaluation setup with no information leakage.

Numerical Instability in Recurrent Networks

Challenge:

Deep recurrent architectures (LSTM/GRU) suffered from vanishing and exploding gradients, leading to unstable training behavior and occasional divergence (NaN losses).

Solution:

A combination of stabilization techniques was applied:

- Gradient clipping (max norm = 1.0)
- Feature standardization (zero mean, unit variance)
- Learning rate scheduling using ReduceLROnPlateau

This ensured stable backpropagation through time and consistent convergence across all models.

Target Distribution Skewness and Spike Preservation

Challenge:

The target variable exhibited strong right skewness with rare but significant high-energy spikes. Conventional outlier removal would have eliminated these critical peak-demand events.

Solution:

All spikes were preserved to maintain real-world signal integrity. To stabilize learning:

- Target scaling was applied using StandardScaler
- Models were trained on normalized targets using MSE loss
- Predictions were inverse-transformed back to Wh for evaluation

This approach preserved peak-demand learning while ensuring stable optimization.

9. Conclusion and Future Work

9.1 Key Findings

- **Temporal learning effectiveness:** Sequence-based models successfully captured temporal dependencies in energy consumption data.
- **Best-performing model:** The GRU model achieved the strongest overall performance with:
 - MAE = **26.47 Wh**
 - RMSE = **57.61 Wh**
- **Architectural efficiency:** GRU provided a strong balance between computational efficiency and predictive accuracy compared to LSTM.
- **Feature engineering impact:** Cyclical time features and lag-based variables significantly improved predictive performance.
- **Tree model competitiveness:** Random Forest performed strongly due to the structured nature of engineered features but was slightly outperformed by GRU.

9.2 Future Work

- **Transformer-based models:** Incorporate attention mechanisms (e.g, Temporal Fusion Transformers) to better capture long-range dependencies.
- **External feature integration:** Include exogenous variables such as occupancy data, weather forecasts, and holiday effects.
- **Multi-step forecasting:** Extend prediction horizon from single-step ($t+1$) to multi-step forecasting ($t+1$ to $t+6$).
- **Probabilistic forecasting:** Predict uncertainty using distributional outputs (e.g, Gaussian or Student-t distributions) for risk-aware energy planning.

10. References

- Candanedo, L. M., Feldheim, V., & Deramaix, D. (2017). *Data-driven prediction models of energy use of appliances in a low-energy house*. *Energy and Buildings*, 140, 81–97.
- Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. *Neural Computation*, 9(8), 1735–1780.
- Cho, K., et al. (2014). *Learning phrase representations using RNN encoder-decoder for statistical machine translation*. arXiv:1406.1078.
- Bergstra, J., & Bengio, Y. (2012). *Random search for hyper-parameter optimization*. *Journal of Machine Learning Research*, 13(3), 281–305.
- Paszke, A., et al. (2019). *PyTorch: An imperative style, high-performance deep learning library*. NeurIPS.

11. Declaration of AI Tools

For this assessment, AI tools (such as GitHub Copilot and Google Gemini) were used as coding assistants. Their role was limited to writing boilerplate code (like setting up command-line arguments and styling the matplotlib plots) and double-checking PyTorch syntax.

All core data science decisions—including the chronological data splitting, handling outlier values, implementing the GRU skip connections, and debugging the sequence windowing alignment—were designed, implemented, and fully understood by myself.